classifier for the sentiment classification problem. You find that your classifier takes a long time to fit the training data. How would you reduce the training time? List all the possible approaches.

7. One of our readers suggested feature scaling/data normalisation as a preprocessing step before you train your model always. Is she correct? Is feature scaling or normalisation always needed in all types of classifiers? Why do you think feature scaling can help achieve faster convergence of your learning procedure? One of the well-known methods of feature scaling is Min-Max scaling. By feature scaling, you are actually throwing away your knowledge of the maximum and minimum values that the feature can take. Wouldn't the loss of this information affect the accuracy of your classifier on unseen data? If you are using a decision tree classifier or random forests, should you still do feature scaling? If yes, explain why.
8. Scikit-learn is a popular machine learning library available in Python, which provides ready-made implementations of several classifiers such as decision tree, support vector machine, random forests, logistic regression, multilayer perceptron, etc. These classifiers provide a 'predict' function, which predicts the output for a given data instance. They also provide a 'predict_proba' function, which returns the probability for each sample (data instance) belonging to a specific output class. For instance, in the case of the movie review sentiment prediction task, with two classes positive and negative, the 'predict_proba' function would return the probability of the sample belonging to the positive sentiment category and negative sentiment category. When would you use the 'predict_proba' function in your sentiment classification task?
9. In the sentiment classification problem on the movie reviews data, you found that some of the reviews did not have the date, country of reviewer and the movie genre. How would you handle these missing data? Note that these features were not numeric; so what kind of data imputation would make sense in this case?
10. In the movie reviews training labelled data set, you are now given certain additional data features that include: (a) the star rating reviewers give to the movie, (b) whether they would like to watch it again, and (c) whether they liked the movie. Would you use these additional features in your training data to train your model? If not, explain why you wouldn't.
11. What is the data leakage problem in machine learning and how do you avoid it? Does the scenario mentioned in question (10) fall under the data leakage category? Detailed information on data leakage and its avoidance can be found in this well-written and must-read paper 'Leakage in data mining: formulation, detection, and avoidance' which was presented at the KDD 2011 conference and is available at *http://dl.acm.org/citation.cfm?id=2020496*.
12. You are using k-fold cross validation for selecting the hyper-parameters of your model. Given that your training data has features which are on widely varying scales, you have decided to do feature scaling. Should you do data scaling once for the entire training data set and then perform the k-fold cross validation? Or should you do the feature scaling within each fold of cross-validation? Explain the reason behind your choice.
13. You are given a data set which has a large number of features. You are told that only a handful of these features are relevant in predicting the output variable. Will you use Lasso regression or ridge regression in this case? Explain the rationale behind your choice. As a follow-up question, when would you prefer ridge regression over Lasso?
14. Decision tree classifiers are very popular in supervised machine learning problems. Two well-known tree classifiers are random forests and gradient boosted decision trees. Can you explain the difference between the two of them? As a follow-up question, can you explain ensemble learning methods in general? When would you opt for an ensemble classifier over a non-ensemble classifier?
15. You are given a data set in which many of the variables are categorical string variables. You decided to encode the categorical variables with One Hot Encoding. Consider that you have a variable called 'country', which can take any of the 20 values. With One Hot encoding, you end up creating 20 new feature variables in place of the single 'country' variable. On the other hand, if you use label encoding, you convert the categorical string variable to a categorical numerical variable. Which of the two methods leads to the 'curse of the dimensionality' problem? When would you prefer to go for One Hot encoding vs label encoding?

Please do send me your answers to the above questions. I will discuss the solutions to these questions in next month's column. I also wanted to alert readers about a new deep learning specialisation course by Prof. Andrew Ng coming up soon on the Coursera platform (*https://www.coursera.org/specializations/deep-learning*). If you are interested in becoming familiar with deep learning, there is no better teacher than Prof. Ng whose machine learning course on Coursera is being taken by more than a million students.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, wishing all our readers wonderful and productive days ahead. END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist at Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=&gid=2339182*